

CREDIT AND WEATHER DERIVATIVES – HOME ASSIGNMENT

Exercise 1

Starting from the Euribor-swap curve data provided, the NSS model is calibrated by defining two functions on Python:

- *nss*, which contain the base formula for the model

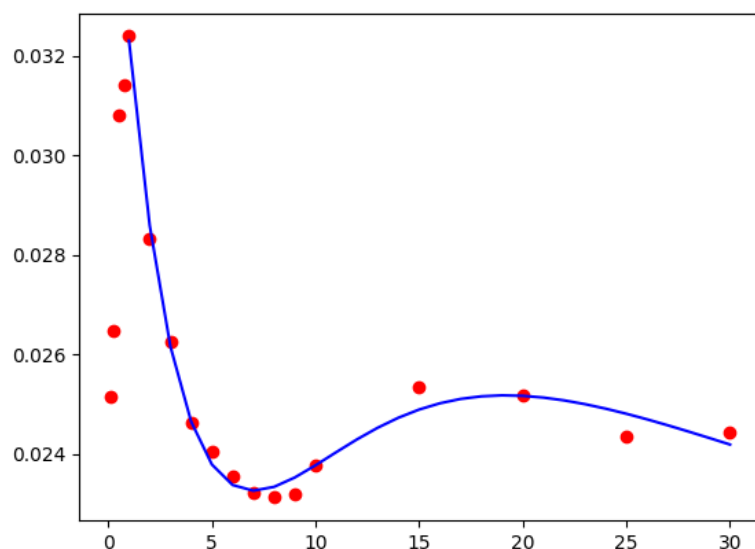
$$s(T) = \beta_0 + \beta_1 \frac{1 - e^{-\frac{T}{\tau_1}}}{\frac{T}{\tau_1}} + \beta_2 \left(\frac{1 - e^{-\frac{T}{\tau_1}}}{\frac{T}{\tau_1}} - e^{-\frac{T}{\tau_1}} \right) + \beta_3 \left(\frac{1 - e^{-\frac{T}{\tau_2}}}{\frac{T}{\tau_2}} - e^{-\frac{T}{\tau_2}} \right)$$

- *residuals*, which compute the errors between observed and predicted values.

The errors are then minimized through the “least_squares” NumPy function with some starting arbitrary parameters, after setting some simple positivity bounds for τ_1 and τ_2 .

After the parameters are calibrated, the NSS curve is calculated for every maturity from 1 to 30 years.

NSS model's parameters	
β_0	0.0178735
β_1	0.0198971
β_2	-2.7095657
β_3	2.6976256
τ_1	4.9683157
τ_2	5.0302482



From the graph of the NSS fitted curve, it's easy to notice that, especially on higher maturities, the model fails to fully explain the variation of the rates over different years (for example, the rise between 25 and 30-years maturity is lost). That is probably due to the limitations imposed by the base formula of the NSS model used, which can't reach certain degrees of precision; for better results, it would probably be better to implement a different version of the formula.

After obtaining the rates for every maturity, the prices of risk-free zero-coupon bonds (or, equivalently, the discount factors) are obtained differently for shorter and longer maturities:

- rates with maturity less than one year are considered risk-free, so DFs are derived directly with the formula:

$$P(0, t) = \frac{1}{1 + R(0, t) \times t}$$

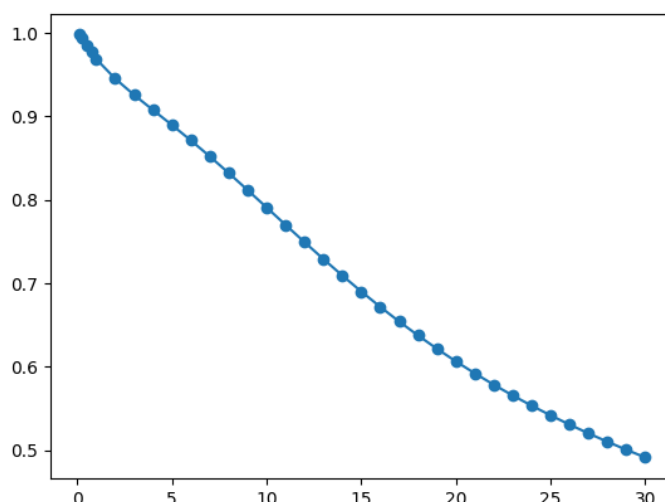
- rates with maturity greater than one year require bootstrapping; so, starting from the first year, the following formula is applied:

$$P(0, t) = \frac{1 - R(0, t) \sum_{k=1}^{t-1} P(0, k)}{1 + R(0, t)}$$

For the first year the calculation is straightforward as the sum on the numerator is equal to zero.

For later maturities, the procedure is repeated in the same way updating the sum's value each time; this was implemented in Python through a simple loop.

The resulting discount factors curve is shown on the graph on the right.



Exercise 2

For this exercise, after importing the data as a Pandas data frame and assuming a recovery rate of 40%, two functions are defined:

- *survival_probs*, which, given a particular term structure of maturities and hazard rates, return the term structure of survival probabilities; this is done reiterating the formula

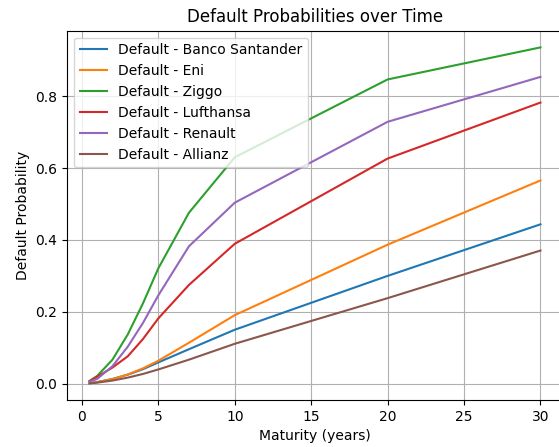
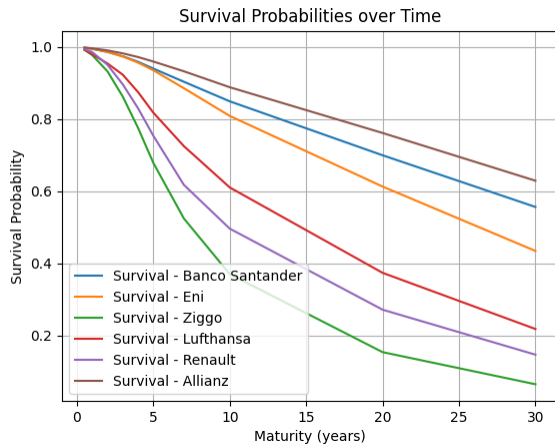
$$Q(0, t) = Q(0, t - 1) \times e^{-\lambda_t dt}$$

and considering a trivial starting value of 1.

- *cds_equation*, which recreates the fair-value equation for a CDS spread and return the difference between the two legs.

Then, for each name and maturity, a root-searching algorithm is launched on the equation, solving it for the unknown parameter λ_t .

Below are presented the graphs for survival and default probabilities over time and the tables containing the hazard rates' estimated values.



Hazard Rates DataFrame:

	Banco Santander	Eni	Ziggo	Lufthansa	Renault	Allianz
0.5	0.004018	0.003259	0.015533	0.015440	0.012156	0.002472
1.0	0.006083	0.005701	0.029161	0.024461	0.015497	0.004242
2.0	0.008593	0.008781	0.047265	0.026490	0.036521	0.005824
3.0	0.012137	0.013050	0.078030	0.033208	0.058685	0.008279
4.0	0.015925	0.017797	0.106171	0.053695	0.076861	0.010513
5.0	0.019736	0.022280	0.133535	0.067428	0.096441	0.012858
7.0	0.019851	0.027402	0.128945	0.060264	0.099643	0.014325
10.0	0.020621	0.030222	0.116148	0.057286	0.072907	0.016184
20.0	0.019380	0.027730	0.087829	0.049095	0.060313	0.015406
30.0	0.022915	0.034430	0.086555	0.053893	0.061544	0.019092

Exercise 3

For this final exercise, for clarity scope, solutions are presented following the same step-by-step structure given in the instructions.

1. The first step consists in applying the given formula, $(1 - e^{-5 \times \frac{CDS_i(t)}{1 - RECI}})$, to each name's time series of CDS 5y spreads. This formula assumes that the hazard rate is constant over time, making it easily derivable from the spread and the recovery rate, here assumed to be 40%.
2. Default probabilities obtained are turned into pseudo-uniform numbers by applying the rank function

$$U_i(t) = \frac{\text{rank}(DP_i(t))}{N + 1}$$

where $U(t) \sim \text{Uniform}(0, 1)$ and N is the number of observations.

3. From the definition of Gaussian copula

$$C_{\Sigma}(u_1, \dots, u_n) = \Phi_{\Sigma}(\Phi^{-1}(u_1), \dots, \Phi^{-1}(u_n))$$

where:

- i. $u_i \sim U(0, 1)$
- ii. Φ^{-1} : inverse standard normal CDF
- iii. Φ_{Σ} : multivariate normal CDF with correlation matrix Σ

The Gaussian copula is fitted by defining $Z_i = \Phi^{-1}(U_i)$, with $Z \sim N(0, \Sigma)$.

The correlation matrix C obtained is shown below.

	ENI	Unicredit	Volkswagen	Allianz	Iberdrola
ENI	1.000000	0.741847	0.848969	0.619041	0.044738
Unicredit	0.741847	1.000000	0.843874	0.723123	0.469740
Volkswagen	0.848969	0.843874	1.000000	0.714370	0.231997
Allianz	0.619041	0.723123	0.714370	1.000000	0.527168
Iberdrola	0.044738	0.469740	0.231997	0.527168	1.000000

4. The Cholesky decomposition matrix A of the obtained correlation matrix C can be found on Python by using the `np.linalg.cholesky()` function from NumPy.

After simulating a large number (here 100.000) of independent random variables $Z_i \sim N(0, 1)$, it's possible to obtain a new random variable $X \sim N(0, C)$ by setting $X = AZ$ with $Z \sim N(0, I)$.

5. After that, a uniform random variable is sampled by setting $U = \Phi(X)$, where Φ is the standard normal CDF.

6. Defining CDS_i as the last spread in the time series for each name, the constant hazard rates are computed, as earlier, dividing this spread by the LGD $(1 - REC)$. Then, from the given formula

$$\{\tau_i > 5\} = \left\{ \int_0^5 \lambda_i dt < -\ln(1 - U_i) \right\}$$

the integral can be approximated to $5\lambda_i$ since intensity is constant. This gives

$$\{\tau_i > 5\} = \{5\lambda_i < -\ln(1 - U_i)\}$$

Since by definition $\tau_i \sim \text{Exp}(\lambda_i)$, $\tau_i = F^{-1}(U_i) = -\frac{\ln(1-U_i)}{\lambda_i}$. After having calculated the default times for every simulation, it's possible to define an indicator function which takes value one if no name defaulted before five years in that simulation, zero otherwise.

Finally, considering the formula for the fair value of the multiname credit derivative given in the exercise,

$$FV = P(0, 5)E^Q \left[\prod_{i=1}^5 1_{\tau_i > 5} \right]$$

given $P(0, 5) = 0.95$ and estimating the expected value in the formula as the average over all the simulations of the indicator function previously defined, a fair value of 0.756789 was obtained.